Multi-Agent RAG on Oracle Cloud — The 4-Day Journey

Four production-style prototypes, each built as a team of AI agents on Oracle Cloud Infrastructure (OCI) with LangGraph. Every prototype retrieves evidence, reranks it, checks its own work, enforces guardrails and pauses for a human before anything consequential. Half taught, half built.

OCI Generative AI · Oracle DB 23ai Vector Search · OCI Rerank · LangGraph multi-agent · Model Context Protocol · Guardrails · Human-in-the-loop · 4 days · 50% taught · 50% built

1 What we are running

- **Format:** a four-day AI-engineering hackathon.
  - **Days 1–3:** teams build a working prototype.
  - **Day 4:** teams present it live and are judged.
- **Platform:** OCI Generative AI (chat, embeddings, rerank), Oracle Database 23ai (tables + vector search), Object Storage, and LangGraph to run the agents.
- **Project family:** all four prototypes are **multi-agent RAG** systems.
  - **RAG** — the system answers by first *retrieving* relevant information, then *generating* an answer grounded in it.
  - **Multi-agent** — instead of one large prompt, several small agents each do one job (plan, search, check, write) and pass a shared state between them.
  - **Agentic** — the team checks its own work, searches again when evidence is weak, pauses for a human when a decision matters, and refuses when a guardrail is tripped.
- **Each team picks one prototype** (A, B, C or D). Each has its own detailed blueprint document.

MANDATORY

Reranker

A second, more precise scoring step after vector search, so agents only see the strongest evidence.

MANDATORY

Guardrails (6 per prototype)

Code-level checks on input, data, generated output and actions — each with a test that proves it works.

MANDATORY

Human-in-the-loop

The graph pauses and waits for a person to clarify, approve, edit or reject before anything consequential.

Optional: conversational memory for follow-up questions, using LangGraph's checkpointer and a thread ID.

2 The four prototypes at a glance

#	Prototype	Business scenario	Talks to…	Signature idea
A	**The Data Concierge** Talk to your Database	UrbanCart, a grocery chain: managers wait 3–5 days for data answers.	An Oracle sales database (11 tables) + schema, glossary and verified-query knowledge bases.	Agents write SQL grounded in retrieved business rules; a code-based SQL Guardian and a human approve it; a Debugger fixes errors; a library of approved queries improves accuracy over time.
B	**The Research Desk** Talk to your Reports	Meridian AMC research desk: 4–6 hours to write each results brief.	72 company PDFs (results, presentations, call transcripts) + Excel fact packs.	Numbers come only from exact table lookups and Python; narrative comes from reranked PDF passages; a Fact-Checker verifies every sentence; a human resolves restated figures.

#	Prototype	Business scenario	Talks to...	Signature idea
C	**The Incident War Room** Talk to your Logs	PayLoom, a payments company: a 2am outage and 40 GB of logs a day.	Synthetic logs from 8 services with planted incidents, a service map and a runbook catalog.	Parallel Log Scouts search each service; a Skeptic agent tries to disprove the Analyst's root cause; fixes are chosen only from approved runbooks and need human approval.
D	**Signal-to-Spec** MCP Server for PMs	Tiffinly, a meal-subscription app: PMs spend a week per quarter reading feedback.	Feedback, usage metrics and backlog, exposed through one **MCP server**.	One server gives any AI client safe tools; a 7-agent client turns feedback into ranked themes, an evidence-backed PRD and tickets; the server itself enforces ticket approval.

3 The shared backbone every prototype uses

All four prototypes follow the same agent pipeline. Only the data, the agent names and a few stage details change.



Retrieval · Reasoning · Guardrail / verification · Human-in-the-loop

Retry loop: step 4 or 7 finds weak evidence → rewrite or widen the search → back to step 2 (always capped)
Clarify: step 1 can pause to ask the human when the question is ambiguous

3.1 How each prototype maps onto the backbone

Step	A · Data Concierge	B · Research Desk	C · Incident War Room	D · Signal-to-Spec
1 Plan	Concierge	Research Lead	Incident Commander	PM Lead
2–3 Retrieve + rerank	Schema Scout over 3 KBs	Numbers Agent + Librarian	Parallel Log Scouts	VoC Researcher via MCP tools
4 Grade	Guardian retry	Evidence Grader	Skeptic "dig deeper"	Theme clustering
6 Generate	SQL Author, Insight Analyst	Writer	Root-Cause Analyst	Prioritiser, PRD Writer
7 Verify	SQL Guardian, Result Checker	Fact-Checker	Skeptic	Evidence Checker
8 Human	Clarify · approve SQL · save verified query	Clarify · resolve conflicts · sign off brief	Clarify · approve runbook · sign off report	Pick themes · review PRD · approve tickets

4 How the 50 / 50 model works

Each build day follows the same rhythm. The facilitator *teaches by building* one part of the system live; teams then build the same part on their own prototype. Nobody watches a finished solution — they watch a part being built, then build it themselves.

Slot	What happens	Who leads
Morning block	Facilitator builds one part live (e.g. the reranker, an approval pause), explaining the concept and the OCI connection.	TAUGHT
Afternoon block	Teams build the same part on their prototype, then the agents marked BUILT in their blueprint.	BUILT
End of day	Every team must pass that day's **checkpoint** — a concrete, verifiable "it works" test.	Both

5 The day-by-day plan

Day 1 — Data, ingestion and the first grounded answer

Goal: generate the synthetic data, load it, and answer one real question from retrieved evidence. No agents yet.

- TAUGHT OCI connections: chat, embeddings, rerank, Oracle 23ai; read-only vs write database users.
- TAUGHT Ingestion: parse → redact personal data → store with metadata → embed. For D: a first MCP server with one tool, tested in MCP Inspector.
- BUILT Teams generate their prototype's synthetic data, **including the planted answer key and traps**, and load it.

> DAY-1 CHECKPOINT
> - Ingestion checkpoint from the team's blueprint passes (e.g. no secrets or personal data in any stored row).
> - One real question gets an answer grounded in retrieved content, with sources printed alongside.

Day 2 — Build the agent team, make it safe

Goal: turn the single pipeline into a team of agents with reranking, guardrails and human pauses.

- TAUGHT The lead agent and LangGraph state; parallel agents with `Send` (C) or tool subsets per agent (D).
- TAUGHT Reranking, with a printed before/after comparison.
- TAUGHT The approval pause with `interrupt` and resume with `Command`.
- BUILT Teams add their retrieval/specialist agents, grading or retry loop, and their first guardrails.

> DAY-2 CHECKPOINT
> The agents visibly:
> - (a) rerank and show the ranking change,
> - (b) retry after weak evidence,
> - (c) block at least one unsafe input and one unsafe output or action,
> - (d) pause for a human before the consequential step.

Day 3 — Verify, evaluate and rehearse

Goal: add the verifier agent, run the evaluation set, fix failures, and rehearse the demo.

- TAUGHT Verifier design (Guardian, Fact-Checker, Skeptic, Evidence Checker) and the evaluation harness.
- BUILT Teams finish their remaining agents and human pauses, run their golden test set, and add optional memory.
- BUILT Full dry-run of the Day-4 demo using the demo script in the blueprint.

> DAY-3 CHECKPOINT
> A full end-to-end run, demo-ready: question → plan → retrieve → rerank → guardrails → generate → verify → human → grounded answer, with nothing hard-coded, plus a filled metrics table.

Day 4 — Presentations and judging

- Each team gives a **live demo** (about 7–8 minutes) following its blueprint's demo script.
- Each team shows its **metrics table** from the golden test set.
- Judging uses the rubric below.

6 Demo and judging rubric

Criterion	What judges look for	Weight
It works, live	A real question answered end-to-end, on the day, on the team's own data.	**20%**
Grounding and verification	Every claim traces to a source (row, page, line ID, feedback ID); the verifier agent visibly catches a bad claim.	**20%**
Retrieval quality	Before/after rerank comparison; retry loop improves weak results.	**15%**

Criterion	What judges look for	Weight
Guardrails	The planted traps are actually blocked when demonstrated.	**15%**
Human-in-the-loop	The pause is real: a human can approve, edit or reject, and it changes the outcome.	**15%**
Engineering clarity	The team can explain each agent's job, the graph, and every OCI service used; metrics are reported honestly.	**15%**

7 Data each team must prepare

All data is **synthetic** and generated by the team on Day 1. Each blueprint gives the exact recipe. The key rule: **plant known answers and traps**, so the team can prove its agents find the right answer and block the wrong actions.

#	What to generate	Planted answers	Planted traps
A	11 retail tables (~1.5M orders) + schema, glossary and 20 example-query entries.	6 business rules (net sales, financial year, weekend, active store...).	PII request, cross-region request, delete request, injection in a product name, huge query.
B	Source JSON for 6 companies × 4 quarters → 72 PDFs + 6 Excel fact packs.	Exact figures for every metric; management commentary per quarter.	Restatement, consolidated vs standalone, forward-looking quote, name clash, injected footnote, missing topic.
C	300k–500k log lines across 8 services over 48 hours + service map + runbooks.	4 incidents with cause chains and red herrings.	Secrets and PII in logs, injection inside a log line, a time with no incident.
D	~40k feedback rows, ~60k daily metric rows, ~1,200 backlog tickets (CSVs) + 3 Markdown resources.	5 feedback themes, one with a matching metric drop.	PII in tickets, injected review, data-export request, duplicate tickets, noise.

8 Prerequisites and pre-work (before Day 1)

> **WHY PRE-WORK MATTERS**
> The biggest risk in a cloud hackathon is losing Day 1 to setup. Every participant should get a working "hello" from an OCI model, an embedding call and a rerank call **before** they arrive.

Access you need

- An **OCI tenancy login**, placed in a **compartment** for the hackathon.
- Access to **OCI Generative AI** in a region that offers it (for example Chicago or Frankfurt), including:
  - a chat model that supports **tool calling** (needed for Prototype D's agents),
  - the embedding model,
  - the **rerank** model — confirm the exact model ID and whether your region needs a dedicated AI cluster for it.
- An **Oracle Database 23ai** (or Autonomous Database) with AI Vector Search, reachable from your laptop, with two users: one read-only, one for loading data.
- An **Object Storage bucket** for raw files.
- An **IAM policy** allowing your group to *use* the generative-ai-family and read the bucket.

Pre-work checklist (do this at home)

#	Step	Done when...
1	Install Python 3.10+ and the packages (below).	`python -c "import langgraph, langchain_oci, mcp"` runs clean.
2	Configure the OCI CLI: `oci setup config` writes `~/.oci/config`.	`oci iam region list` returns data.

#	Step	Done when…
3	Run the "hello model" snippet (below).	You get a sentence back from an OCI LLM.
4	Run one embedding call and one rerank call.	You get a vector back, and a ranked list of 3 test documents.
5	Connect to Oracle DB 23ai with `python-oracledb`, as both users.	`SELECT 1 FROM dual` works for both; the read-only user cannot `CREATE TABLE`.
6	Prototype D teams: install Node.js 18+ for the MCP Inspector.	`npx @modelcontextprotocol/inspector` opens in the browser.
7	Note your **compartment OCID**, **region endpoint** and **DB connection strings**.	All saved where you can copy them.

INSTALL ONCE

```
pip install langgraph langchain-core langchain-oci langchain-oracledb langchain-text-splitters \
            oci python-oracledb pandas openpyxl pdfplumber reportlab sqlglot scikit-learn \
            mcp langchain-mcp-adapters
```

HELLO, OCI MODEL — THE ONE SNIPPET EVERYONE RUNS BEFORE DAY 1

```
from langchain_oci import ChatOCIGenAI

llm = ChatOCIGenAI(
    model_id="meta.llama-3.3-70b-instruct",
    service_endpoint="https://inference.generativeai.us-chicago-1.oci.oraclecloud.com",
    compartment_id="ocid1.compartment.oc1..xxxxxxxx",   # your compartment OCID
    model_kwargs={"temperature": 0, "max_tokens": 256},
)
print(llm.invoke("Say hello from Oracle Cloud in one sentence.").content)
```

Top setup gotchas

- **Wrong region** — Generative AI, and rerank in particular, is only in certain regions. The `service_endpoint` region must host the models you use.
- **Missing IAM policy** — "NotAuthorized" almost always means the policy to use generative-ai-family is missing.
- **Model ID typo** — model IDs change over time. Confirm exact IDs in the OCI console's Generative AI model list.
- **Rerank not available** — every blueprint gives a fallback: ask the chat model to score each candidate 0–10 and keep the top ones.
- **Deprecated import** — use `langchain-oci`, not the old `langchain-community` OCI classes.

9 The shared toolbox

Service / library	Role	How you reach it
OCI Generative AI — chat	Powers every agent's reasoning and writing.	`langchain_oci.ChatOCIGenAI`
OCI Generative AI — embeddings	Turns text into vectors for search.	`langchain_oci.OCIGenAIEmbeddings`
OCI Generative AI — rerank	Re-scores search candidates against the question.	`oci.generative_ai_inference` (`rerank_text`)
Oracle DB 23ai	Business tables, raw records, vector stores, audit logs.	`python-oracledb`, `langchain_oracledb.OracleVS`
Object Storage	Raw files before ingestion.	`oci.ObjectStorageClient`
LangGraph	Runs the agents: state, edges, loops, parallel branches, human pauses, memory.	`StateGraph`, `Send`, `interrupt`, `Command`

Service / library	Role	How you reach it
MCP (Prototype D)	Exposes tools, resources and prompts to any AI client.	`mcp.server.fastmcp.FastMCP`, `langchain_mcp_adapters`
Helpers	SQL safety parsing (A), PDF/Excel reading (B), clustering (D).	`sqlglot`, `pdfplumber`, `pandas`, `scikit-learn`

10 Plain-English glossary

Term	In one line
Embedding	A list of numbers that captures the meaning of a piece of text, so texts with similar meanings get similar numbers.
Vector search	Finding stored text whose embedding is closest to the question's embedding. Fast but rough.
Reranker	A model that reads the question and each candidate together and gives a precise relevance score. Slower, so it runs only on the top candidates.
RAG	Retrieve first, then generate an answer grounded in what was retrieved.
Agent	A model call (often with tools) that has one job, one prompt, and reads/writes the shared state.
Multi-agent	Several agents with different jobs, connected by a graph that decides who runs next.
Verifier agent	An agent whose only job is to check another agent's output against the evidence (Guardian, Fact-Checker, Skeptic, Evidence Checker).
State	The shared record passed from agent to agent (question, evidence, drafts, decisions).
Human-in-the-loop	The graph stops and waits for a person to clarify, approve, edit or reject before continuing.
Guardrail	An automatic check on input, data, output or action that can block, redact or refuse.
Grounding	Every claim in the answer can be traced to a retrieved source.
MCP	Model Context Protocol — an open standard for exposing tools, resources and prompts from a server to any AI client.
Planted answer key	Known facts and traps deliberately put into synthetic data, so results can be measured.

HOW TO READ THE REST OF THE SET

Blueprints A, B, C and D each cover one prototype in full, in the same order: business scenario → what we will build → sample conversation → requirements → data with planted answers → architecture and agent roster → OCI setup → ingestion → retrieval and reranking → agents one by one → LangGraph orchestration → guardrails → human-in-the-loop → evaluation → 3-day build plan → demo script → stretch goals and troubleshooting.